

面向对象编程（Python）

1.基础语法

2.面向对象编程

1.基础语法

```
1  1.中文编码
2  文字开头加入
3  # -*- coding: UTF-8 -*-解决
4
5  2.基础语句
6  >>>print ("hello")
7  #不需要分号换行
8  if
9
10     else:
11     if
12     else:
13     else:
14  #缩进表示逻辑不可省去
15
16  3.变量类型
17  >>>list:
18  number
19  string
20  list
21  tuple
22  dictionary
23
24  number支持: int float long complex,复数
25
26  string: "abcdef"
27  索引: s1[1: 5]="bcdef"
28  s1="123456"
29  s2="7890"
30  s3=s1+s2="1234567890"
31
32  list: list1=[a,b,c,d,e]
33  索引同上
34  列表允许更新, 允许对元素重新赋值
35
36  tuple: tuple1=(a,b,c,d,e)
37  元组不允许更新, 不能重新赋值, 但能连接
38  as: tuple3=tuple1+tuple2
39
40  dictionary: 由索引key和值value组成
41
42  #创建空典
43  dict={}
44  #添加元素
45  dict['one']="this is one"
```

```

46 dict[2]="this is 2"
47 #输出完整字典
48 print dict
49
50 4.运算符:
51 算术运算符pass
52 比较运算符pass
53 赋值运算符pass a **=c 幂指数运算
54 逻辑运算符pass
55 位运算符 pass
56 成员运算符(in/not in): return Ture / Flase
57
58 5.循环语句
59 while a<10:
60     print(a)
61 else: #循环结束可用else
62
63 for ar in sequence:
64     statements(ar)
65 #迭代执行
66 fruit = ['a','b','c']
67 for index in range(len(fruits)):
68     print(fruit[indix])
69 else: #循环结束可用else
70
71     break/continue/pass语句
72
73 6.python.function
74
75 def functionname(parameters)
76     "documents"
77     statement()
78     return
79 可变与不可变对象，即传入为型参还是实参
80 tuple/number/string为不可变对象，传入为形参
81 list/dictionary
82
83 a.定义必备参数与不定长参数:
84 def functionname(str)
85     print str
86     return [expresssion]
87 调用:
88 必备参数: functionnname(a)
89 关键词参数: fcname(str="my string")
90 默认参数: 跟缺省定义一样
91 不定长参数:
92 def function(arg1,*vartuple)
93     statements(arg)

```

```

94     .....
95     return
96
97     其中所有不定长的参数将被存储在artuple中使用
98
99     7.module
100
101     from module import a,b,c,d...
102     在module中引入abdc...
103     引入所有用*替代

```

2.面向对象编程

- 类(Class): 用来描述具有相同的属性和方法的对象的集合。它定义了该集合中每个对象所共有的属性和方法。对象是类的实例
- #class
- 类变量：类变量在整个实例化的对象中是公用的。类变量定义在类中且在函数体之外。类变量通常不作为实例变量使用。
- #类里面的私有/公有变量
- 数据成员：类变量或者实例变量, 用于处理类及其实例对象的相关的数据。
- #express exactly
- 方法重写：如果从父类继承的方法不能满足子类的需求，可以对其进行改写，这个过程叫方法的覆盖（override），也称为方法的重写。
- #方法复用/覆盖
- 局部变量：定义在方法中的变量，只作用于当前实例的类。
- #class里函数定义所用的变量
- 实例变量：在类的声明中，属性是用变量来表示的。这种变量就称为实例变量，是在类声明的内部但是在类的其他成员方法之外声明的。
- #创建的类实例变量
- 继承：即一个派生类（derived class）继承基类（base class）的字段和方法。继承也允许把一个派生类的对象作为一个基类对象对待。例如，有这样一个设计：一个Dog类型的对象派生自Animal类，这是模拟"是一个（is-a）"关系（例图，Dog是一个Animal）。
- 实例化：创建一个类的实例，类的具体对象。
- 创建类实例
- 方法：类中定义的函数。

- express exactly
- 对象：通过类定义的数据结构实例。对象包括两个数据成员（类变量和实例变量）和方法。
- express exactly

```

1  . 创建类:
2
3  class Classname:
4      <变量声明>
5
6
7      #__表示私有, 不可从外部访问, 功能近似于c++的构造函数
8      #self表示的是类的实例, 而非类的定义
9
10     def __init__(self,name,salary)
11         self.name = name
12         <self中元素的初始化>
13
14     def function1(self):
15         statements...
16     def function2(self):
17         statements...
18
19  创建类的实例:
20  class1 = Classname(name1,salary1)
21  #创建第一个类的实例, 参数为构造函数中的定义参数
22
23
24  2. 访问属性
25  emp1.displayEmployee()
26  emp2.displayEmployee()
27  可以添加, 删除, 修改类的属性, 如下所示:
28  emp1.age = 7 # 添加一个 'age' 属性
29  emp1.age = 8 # 修改 'age' 属性
30  del emp1.age # 删除 'age' 属性
31
32  Python内置类属性
33  __dict__ : 类的属性 (包含一个字典, 由类的数据属性组成)
34  __doc__ : 类的文档字符串
35  __name__ : 类名
36  __module__ : 类定义所在的模块 (类的全名是 '__main__.className', 如果类位于一个导入模块mymod中, 那么className.__module__ 等于 mymod)
37  __bases__ : 类的所有父类构成元素 (包含了一个由所有父类组成的元组)
38
39  3. 类的继承
40  class Parent: # 定义父类
41      parentAttr = 100

```

```

42     def __init__(self):
43         print "调用父类构造函数"
44
45     def parentMethod(self):
46         print '调用父类方法'
47
48     def setAttr(self, attr):
49         Parent.parentAttr = attr
50
51     def getAttr(self):
52         print "父类属性 :", Parent.parentAttr
53
54 class Child(Parent): # 定义子类
55     def __init__(self):
56         print "调用子类构造方法"
57
58     def childMethod(self):
59         print '调用子类方法'
60
61 #####
62 关于构造函数，如果子类不重写__init__，就直接使用父类构造
63 如果重写了__init__ 时，要继承父类的构造方法，可以使用 super 关键字：（重写一部分）
64 class Son(Father):
65     def __init__(self, name):
66         super(Son, self).__init__(name)
67         print ("hi")
68         self.name = name
69 继承父类但重写
70 ###
71 方法重写（函数覆盖，子类覆盖父类）
72 #!/usr/bin/python
73 # -*- coding: UTF-8 -*-
74
75 class Parent:          # 定义父类
76     def myMethod(self):
77         print '调用父类方法'
78
79 class Child(Parent): # 定义子类
80     def myMethod(self):
81         print '调用子类方法'
82
83 c = Child()             # 子类实例
84 c.myMethod()           # 子类调用重写方法
85
86 4. 类属性与方法
87 私有变量与公共变量
88 class JustCounter:
89     __secretCount = 0 # 私有变量

```

```
90     publicCount = 0    # 公开变量
91
92     def count(self):
93         self.__secretCount += 1
94         self.publicCount += 1
95         print self.__secretCount
96
97 counter = JustCounter()
98 counter.count()
99 counter.count()
100 print counter.publicCount
101 print counter.__secretCount # 报错，实例不能访问私有变量
```